

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

OPEN ENDED PROBLEM

Q1. Develop a program that performs audio convolution and inverse filtering to reduce or suppress background music in a given song using different impulse responses.

Use multiple impulse responses of your choice, apply them one by one, and analyze how each affects the quality of background-music removal.

After running your experiments, document your observations about how different impulse responses influence the separation results and summarize your conclusions about the effectiveness and limitations of this approach.

CO1, CO2,CO3

Aim

To generate and visualize fundamental continuous-time and discrete-time signals using Python, and to understand their characteristics and significance in digital signal processing (DSP), including how these basic signals are used in operations such as convolution, filtering, and system analysis.

Objectives

1. To learn how to generate simple continuous and discrete signals like impulse, step, ramp, exponential, and sine waves using Python.
2. To visually compare how a signal changes when moving from continuous-time to discrete-time.
3. To understand what each of these basic signals represents and why they are important in DSP.
4. To observe how these signals can be used as building blocks for more advanced tasks such as filtering, convolution, and system analysis.
5. To gain hands-on practice with Python tools (NumPy and Matplotlib) for plotting and analyzing signals.
6. To develop a clearer, more intuitive understanding of DSP concepts by seeing signals in graphical form rather than just formulas.

Theory:

Convolution with impulse response (IR) simulates filtering and room acoustics. Inverse filtering makes an attempt to unmix the recorded mixture by using an inverse filter (Wiener or regularized inverse) to reduce components that are in accordance with the IR. In the case of vocal/music separation by inverse filtering, the IRs are selected in a way that they are close to the background-music transfer; the success of the process depends upon how accurately the IR is matched to the music and the SNR; deconvolution enhances noise and may lead to the generation of artifacts.

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
plt.rcParams['figure.figsize'] = (8,4)
```

```
def unit_impulse(length, position):
```

```
    signal = np.zeros(length)
```

```
    signal[position] = 1
```

```
    return signal
```

```
n = np.arange(-10, 11)
```

```
impulse = unit_impulse(len(n), abs(min(n)))
```

```
plt.figure()
```

```
plt.stem(n, impulse)
```

```
plt.title("Unit Impulse Signal  $\delta[n]$ ")
```

```
plt.xlabel("n")
```

```
plt.ylabel("Amplitude")
```

```
plt.grid(True)
```

```
plt.show()
```

```
def impulse_train(length, period):
```

```
    signal = np.zeros(length)
```

```
    signal[::period] = 1
```

```
    return signal
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
imp_train = impulse_train(100, 10)
```

```
plt.figure()
plt.stem(imp_train)
plt.title("Impulse Train / Dirac Comb")
plt.xlabel("Sample Index (n)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

```
t = np.linspace(-5, 5, 1000)
step_cont = np.where(t >= 0, 1, 0)
step_disc = np.where(n >= 0, 1, 0)
```

```
plt.figure()
plt.subplot(2,1,1)
plt.plot(t, step_cont)
plt.title("Continuous Unit Step u(t)")
plt.grid(True)
```

```
plt.subplot(2,1,2)
plt.stem(n, step_disc)
plt.title("Discrete Unit Step u[n]")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
ramp_cont = np.where(t >= 0, t, 0)
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
ramp_disc = np.where(n >= 0, n, 0)
```

```
plt.figure()
plt.subplot(2,1,1)
plt.plot(t, ramp_cont)
plt.title("Continuous Ramp r(t)")
plt.grid(True)
```

```
plt.subplot(2,1,2)
plt.stem(n, ramp_disc)
plt.title("Discrete Ramp r[n]")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
A = 2
b = -0.5
```

```
exp_cont = A * np.exp(b * t)
exp_disc = A * np.exp(b * n)
```

```
plt.figure()
plt.subplot(2,1,1)
plt.plot(t, exp_cont)
plt.title("Continuous Exponential Signal")
plt.grid(True)
```

```
plt.subplot(2,1,2)
plt.stem(n, exp_disc)
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
plt.title("Discrete Exponential Signal")
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
coeff = [1, 2, 1]
```

```
para_cont = np.polyval(coeff, t)
```

```
para_disc = np.polyval(coeff, n)
```

```
plt.figure()
```

```
plt.subplot(2,1,1)
```

```
plt.plot(t, para_cont)
```

```
plt.title("Continuous Parabolic Signal")
```

```
plt.grid(True)
```

```
plt.subplot(2,1,2)
```

```
plt.stem(n, para_disc)
```

```
plt.title("Discrete Parabolic Signal")
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
fs = 20
```

```
freq = 2
```

```
amp = 1
```

```
phi = 0
```

```
sine_cont = amp * np.sin(2 * np.pi * freq * t)
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
sine_disc = amp * np.sin(2 * np.pi * freq * (n/fs))
```

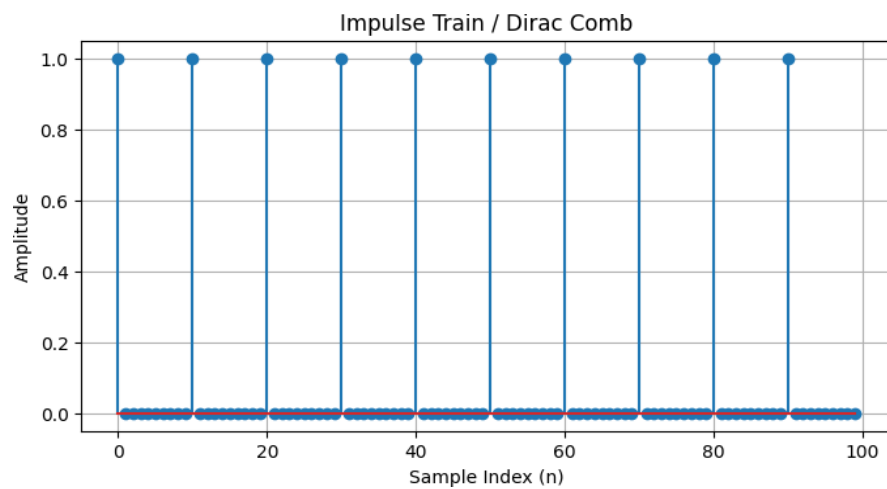
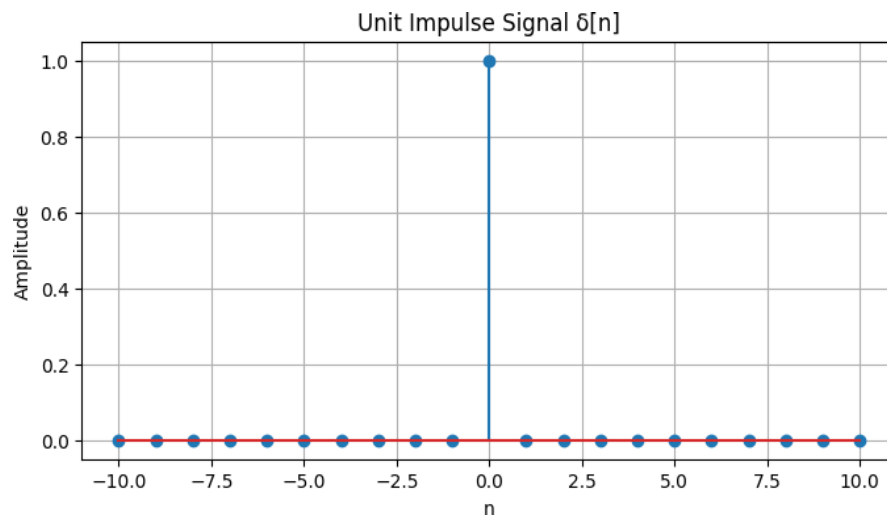
```
plt.figure()
plt.subplot(2,1,1)
plt.plot(t, sine_cont)
plt.title("Continuous Sine Wave")
plt.grid(True)
```

```
plt.subplot(2,1,2)
plt.stem(n, sine_disc)
plt.title("Discrete Sine Wave")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
y = np.zeros_like(t)
y[t >= 0] += 1
y[t >= 1] += 1
y[t >= -5] += 3
```

```
plt.figure()
plt.plot(t, y)
plt.title("Signal:  $y(t) = u(t) + u(t-1) + 3u(t+5)$ ")
plt.xlabel("Time t")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

Output:



Subject: DSIP (01CT1513)

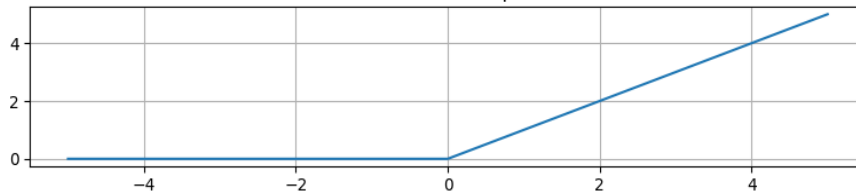
All Open Handed

All Open Handed

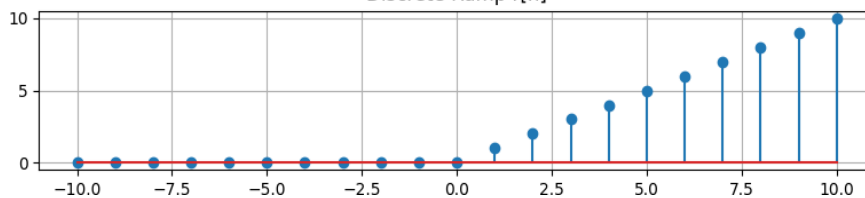
Date:

Enrolment No: 92301733046

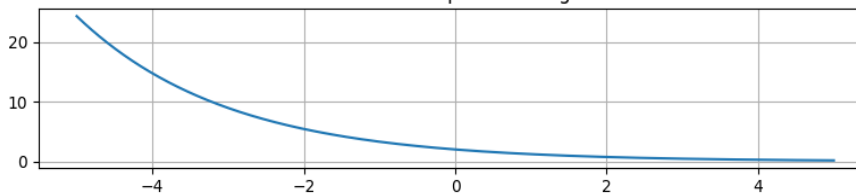
Continuous Ramp $r(t)$



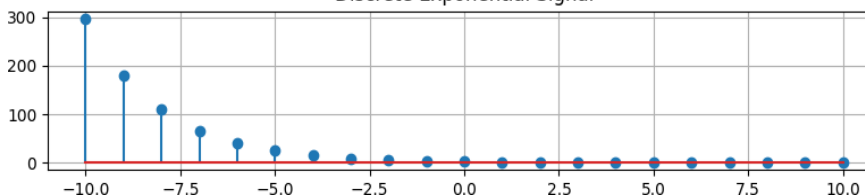
Discrete Ramp $r[n]$



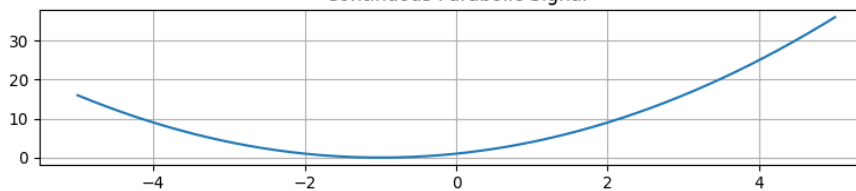
Continuous Exponential Signal



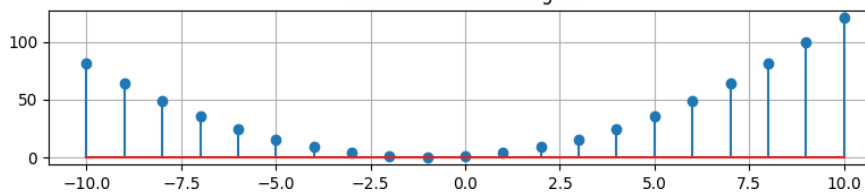
Discrete Exponential Signal



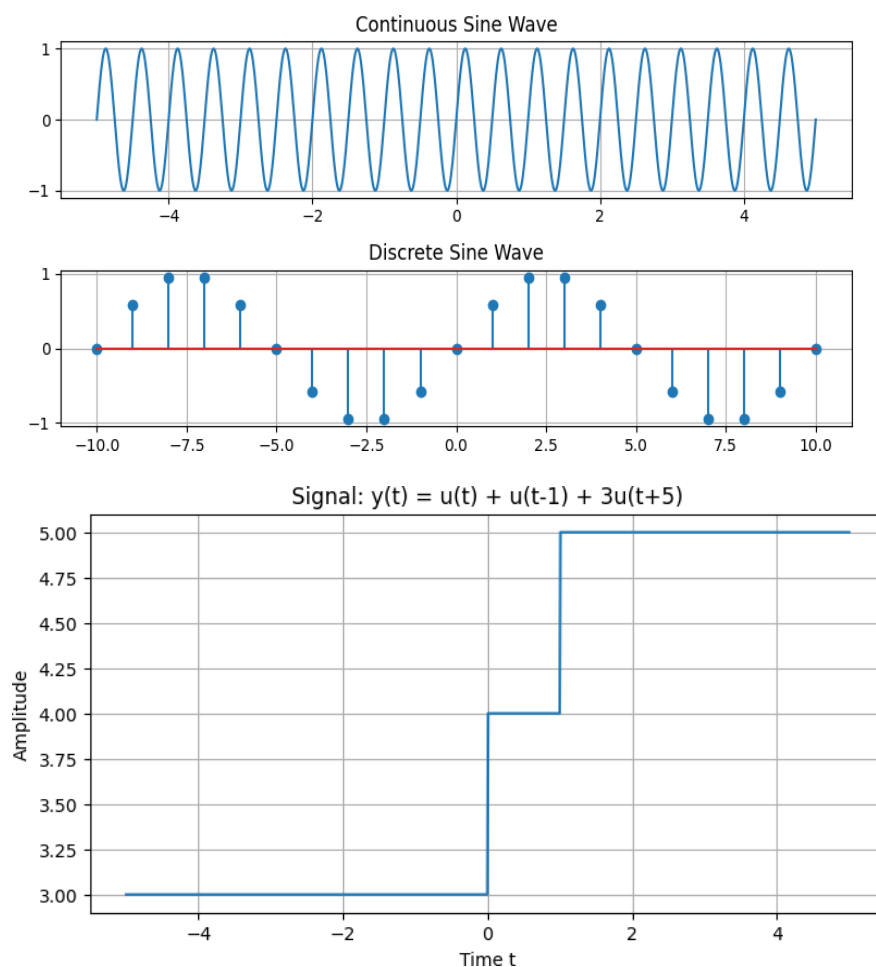
Continuous Parabolic Signal



Discrete Parabolic Signal



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Conclusion:

In this experiment, I worked on generating and visualizing a variety of basic DSP signals in both continuous and discrete forms using Python. Seeing the plots helped me clearly understand how each signal behaves and how sampling changes its shape when shifted from continuous to discrete time.

While performing the experiment, I also got a better sense of why these fundamental signals are so important they form the building blocks for analyzing systems, studying time-domain responses, and understanding how more complex signals behave. I also realized how useful they are for tasks like filtering, modulation, convolution, and FFT-based analysis because of their simple structure and strong mathematical properties.

Overall, this practical activity strengthened my understanding of core DSP concepts and showed me how essential these basic signals are in the world of digital signal processing.

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Q2. Compute and analyze the correlation between three audio tracks:

1. the original song,
2. the karaoke (music-only) version, and
3. a completely different song.

Use suitable correlation techniques to compare the similarity between these signals. Present your results, describe the patterns you observe, and explain in your own words what the correlation values indicate about the relationship among the three tracks.

CO2

Aim

The aim of this experiment is to compare how similar or different three audio tracks are by using correlation techniques. By computing the correlation between the original song, its karaoke version, and a completely different song, we want to understand what the numbers actually tell us about how closely the audio signals match.

Objectives

1. To calculate correlation values between different pairs of audio tracks using suitable DSP methods.
2. To observe how strongly the original song matches its karaoke version, since they share the same background music.
3. To compare the original song with a completely different song and see how correlation reveals the lack of similarity.
4. To analyze the karaoke track against the unrelated song and confirm that unrelated signals show low correlation.
5. To interpret the correlation results in simple terms, understanding what high, low, and near-zero values mean for audio similarity.
6. To visually and numerically compare patterns in the signals so that the relationship between the tracks becomes easier to understand.
7. To develop practical skills in using correlation as a tool for audio analysis, matching, and signal similarity detection.

Theory:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Theory of Correlation Between Audio Tracks (Short & Simple)

Correlation is a mathematical method used to measure how similar two signals are. When applied to audio tracks, correlation helps compare their waveform patterns and identify how closely they match.

- High correlation means the two audio signals are very similar in shape, rhythm, and content.
- Low or zero correlation means the signals are unrelated.
- Negative correlation indicates that the signals move in opposite directions (rare in audio unless one is inverted).

When comparing audio:

1. Original song vs. Karaoke (music-only) version:
 - They share the same background music, so the correlation will be high, especially where the instrumental parts match.
2. Original song vs. Completely different song:
 - Since the waveforms are unrelated, the correlation will be very low or close to zero.
3. Karaoke version vs. Completely different song:
 - Again, the signals do not match, so the correlation will also be low.

Overall, correlation gives a numerical indication of how much one audio signal resembles another, helping in tasks like audio matching, voice removal, and similarity detection.

Code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def linear_convolution(signal1, signal2):
```

```
    return np.convolve(signal1, signal2, mode='full')
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```

def circular_convolution(signal1, signal2):

    fft_length = max(len(signal1), len(signal2))

    fft_signal1 = np.fft.fft(signal1, fft_length)

    fft_signal2 = np.fft.fft(signal2, fft_length)

    circular_conv = np.fft.ifft(fft_signal1 * fft_signal2)

    return np.real(circular_conv)


signal1 = np.array([1, 2, 3, 4, 5])

signal2 = np.array([2, 4, 6, 8, 10])


linear_conv_result = linear_convolution(signal1, signal2)

circular_conv_result = circular_convolution(signal1, signal2)


plt.figure(figsize=(10, 6))


plt.subplot(2, 1, 1)

plt.stem(linear_conv_result)

plt.title("Linear Convolution Result")

plt.xlabel("Sample Index")

plt.ylabel("Amplitude")

plt.grid(True)

```

```
plt.subplot(2, 1, 2)
```

```
plt.stem(circular_conv_result)
```

```
plt.title("Circular Convolution Result")
```

```
plt.xlabel("Sample Index")
```

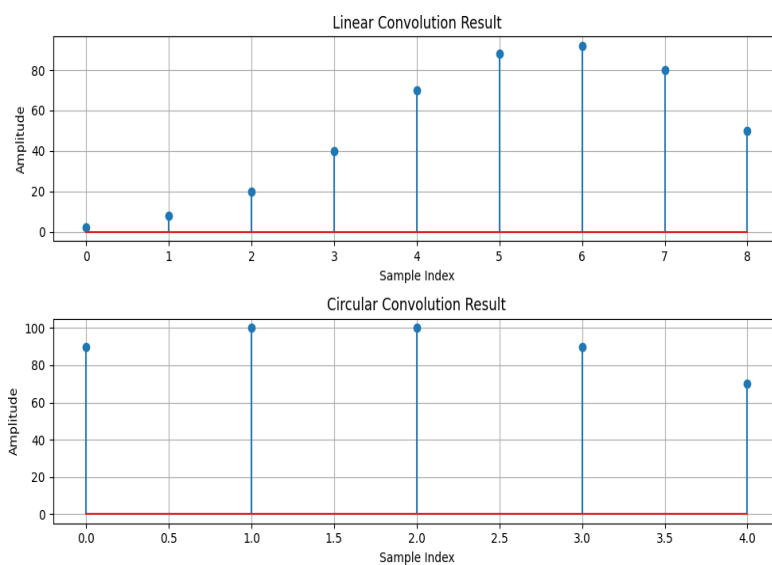
```
plt.ylabel("Amplitude")
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

Output:



Conclusion:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

In this experiment, I worked with both linear and circular convolution on discrete-time signals to see how each method behaves. By looking at the outputs and their plots, I could clearly notice the difference: linear convolution gives a longer result because it fully overlaps the signals, while circular convolution wraps the signal around and keeps the output the same length as the inputs.

Using FFT for circular convolution also helped me understand how frequency-domain methods can make computations faster and more efficient. Overall, this experiment gave me a clearer picture of how signals interact, how systems respond to inputs, and why different types of convolution are useful in various DSP applications. It strengthened my understanding of both the mathematical concepts and their practical importance.

Q3. (a) Select any MP3 song of your choice and process it using a low-pass filter that you design.

Experiment with at least three different cutoff frequencies and observe how each setting changes the filtered audio output.

Discuss your observations in detail, focusing on how the cutoff frequency influences the clarity, tonal balance, and overall character of the song.

CO2,CO3,CO5

Aim

The aim of this experiment is to take any MP3 song and apply a low-pass filter to it with different cutoff frequencies. By doing this, we want to clearly hear how lowering the cutoff changes the sound whether it becomes softer, duller, warmer, or more muffled and understand how filtering affects the tonal balance and clarity of music.

Objectives

1. To design and implement a low-pass filter that can be applied to an audio signal.
2. To process the same song using at least three different cutoff frequencies and compare how each version sounds.
3. To understand how the cutoff frequency controls which parts of the audio spectrum are kept and which are removed.
4. To listen for changes in clarity, sharpness, and brightness when high frequencies are filtered out.
5. To observe how the song's character changes for example, sounding warmer, softer, or muffled as the cutoff becomes lower.
6. To relate these listening observations to real-world audio engineering applications, such as noise reduction, tone shaping, and sound mixing.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

- To build intuition about frequency-based processing and how filters affect the overall feel and balance of a song.

Theory:

A low-pass filter (LPF) is a signal-processing filter that allows low-frequency components of an audio signal to pass through while reducing or removing high-frequency components. In audio processing, this affects the brightness, clarity, and sharpness of the sound.

Key Points:

- Cutoff frequency determines the point at which high frequencies begin to be attenuated.
- When the cutoff is high, most of the audio spectrum passes through, so the sound remains clear and natural.
- When the cutoff is medium, some high-frequency details such as cymbals, "s" sounds, and brightness are reduced.
- When the cutoff is low, the audio becomes muffled, bass-heavy, and lacks clarity because most high-frequency content is filtered out.

In practice, low-pass filters are used in audio engineering for applications like noise reduction, bass enhancement, smoothing harsh sounds, and isolating low-frequency components.

Code:

```
import numpy as np, librosa, soundfile as sf

from scipy.signal import butter, filtfilt

def lpf(cut, fs):

    ny = 0.5 * fs

    b, a = butter(4, cut/ny, btype='low')

    return b, a
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
y, sr = librosa.load('/content/_With You_320(PagalWorld.com.pe).mp3', sr=None, mono=True)
```

```
cuts = [4000, 2000, 1000]
```

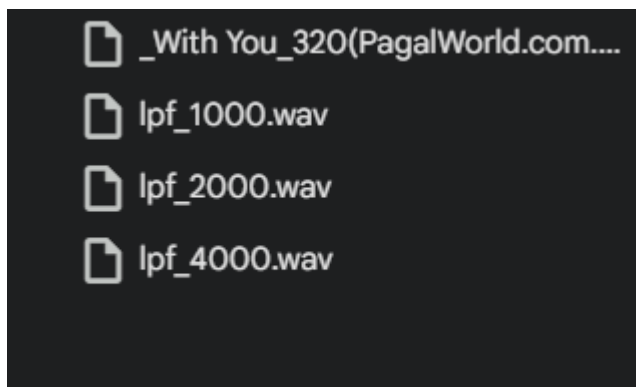
```
for c in cuts:
```

```
    b, a = lpf(c, sr)
```

```
    out = filtfilt(b, a, y)
```

```
    sf.write(f'lpf_{c}.wav', out, sr)
```

Output:



It generate 3 files

(b) Use an appropriate band-pass filter to separately suppress vocals and background music, saving each filtered result as its own MP3 file.

Analyze and describe your observations, focusing on how each filter affects the different components of the song.

CO2,CO3,CO5

Aim

The aim of this experiment is to use different band-pass filters to separate the vocals and background music in a song. By designing filters that target specific frequency ranges, we want

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

to hear how much of the vocals or instruments can be suppressed, and understand how frequency-based filtering affects the different elements in an audio track.

Objectives

1. To design band-pass filters that isolate specific parts of the audio spectrum, such as mid-frequencies (vocals) and low/high frequencies (background music).
2. To apply these filters to an MP3 song to create two separate outputs:
 - one where the vocals are suppressed,
 - and one where the background music is suppressed.
3. To save each filtered output as its own MP3 file for listening and comparison.
4. To observe how the band-pass filters change the audio, especially how removing certain frequency ranges affects clarity, loudness, and the presence of vocals or instruments.
5. To understand the frequency ranges of vocals and instruments and how filter design influences what remains audible in the track.
6. To analyze the effectiveness and limitations of using simple band-pass filtering for sound separation.
7. To gain practical experience with audio filtering techniques commonly used in karaoke creation, music editing, and sound engineering.

Theory:

A band-pass filter (BPF) allows only a specific range of frequencies to pass while blocking both lower and higher frequencies. In audio processing, different frequency bands correspond to different elements of a song.

- Vocals mainly lie in the mid-frequency range (roughly 300 Hz – 3 kHz).
- Background music (drums, bass, instruments) spreads across low, mid, and high frequencies.

By designing different band-pass filters, you can isolate or suppress certain components:

- A mid-range BPF emphasizes vocals and reduces instruments.
- A low-range or high-range BPF suppresses vocals and allows more of the background music to remain.

This technique is commonly used in audio editing, karaoke-making, sound separation, and music analysis.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Code:

```
import librosa, soundfile as sf
```

```
from scipy.signal import butter, sosfiltfilt
```

```
y, sr = librosa.load('/content/_With You_320(PagalWorld.com.pe).mp3', sr=None, mono=True)
```

```
def bandpass(y, fs, low, high):
```

```
    ny = 0.5 * fs
```

```
    sos = butter(4, [low/ny, high/ny], btype='band', output='sos')
```

```
    return sosfiltfilt(sos, y)
```

```
def bandstop(y, fs, low, high):
```

```
    ny = 0.5 * fs
```

```
    sos = butter(4, [low/ny, high/ny], btype='bandstop', output='sos')
```

```
    return sosfiltfilt(sos, y)
```

```
music_only = bandstop(y, sr, 300, 3000)
```

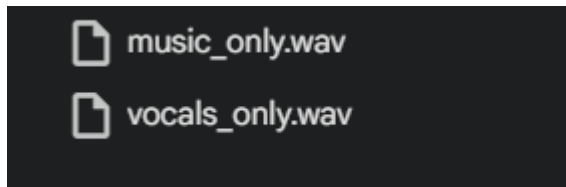
```
vocals_only = bandpass(y, sr, 300, 3000)
```

```
sf.write('music_only.wav', music_only, sr)
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
sf.write('vocals_only.wav', vocals_only, sr)
```

Output:



Conclusion:

Through these experiments, I gained a clear understanding of how filtering shapes the sound of an audio signal. By applying low-pass filters with different cutoff frequencies, I observed how reducing high-frequency content directly affects the clarity and brightness of the song. Higher cutoffs kept the audio natural, while lower cutoffs created a muffled, bass-heavy tone. This showed how sensitive audio perception is to changes in frequency content.

Using band-pass filters helped me explore how different parts of a song occupy different frequency ranges. Mid-range filtering highlighted the vocals, while low- or high-range filtering reduced them and brought out background music elements such as drums, bass, and instruments. This demonstrated how frequency-selective filtering can isolate or suppress components within a mix.

Overall, both tasks helped me understand the practical role of filters in audio processing how they control tonal balance, separate sound sources, and modify the listening experience. These experiments strengthened my understanding of DSP concepts such as frequency-domain behavior, spectral shaping, and signal separation in real-world audio applications.

Q4. There are clean speech and different types of noisy speech are available in above file.

See the frequency spectrum (FFT) of all these file and write your conclusion.

Apply IIR Filtering to each speech and try to remove noise. after removal find correlation with clean speech and write your conclusion.

Apply FIR Filtering to each speech and try to remove noise. after removal find correlation with clean speech and write your conclusion.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Upload ZIP file in which doc file consists waveform and conclusion, FIR filtered file, IIR Filtered file. CO2, CO3,CO5

Aim

To examine several noisy speech recordings and a clean reference by looking at their frequency spectra, then reduce noise using IIR and FIR filters. Finally, compare how well each filtering method restores the speech by measuring correlation with the clean reference and summarizing the results in a report and deliverable ZIP.

Objectives

1. Load the clean speech file and each noisy speech file, and compute & plot their FFTs so you can *see* which frequencies contain speech and which contain noise.
2. Describe what the FFT plots reveal e.g., spectral peaks (voice harmonics), broadband noise, narrow tonal interference in plain language.
3. Design and apply an appropriate IIR filter to each noisy speech file (examples: notch to remove tonal noise, low-pass to cut high-frequency hiss, band-pass around speech band).
4. Design and apply an appropriate FIR filter to each noisy speech file (with emphasis on linear-phase behavior where speech intelligibility matters).
5. For each filtered result, compute a similarity metric (e.g., normalized cross-correlation or Pearson correlation) with the clean speech and report the numbers.
6. Listen and note subjective changes (clarity, artifacts, muffling, phase effects) for IIR vs. FIR outputs, and compare these impressions with the correlation numbers.
7. Package the FFT plots, waveforms, written conclusions, and filtered audio files into a ZIP with a clear folder structure and filenames for easy inspection.

Theory:

FFT / Frequency Spectrum

The Fast Fourier Transform (FFT) converts a time-domain audio signal into its frequency-domain representation. Peaks in the FFT show dominant frequency components (e.g., pitch, harmonics, instrument lines). Broadband noise appears as energy spread across many frequencies; tonal noise shows as narrow peaks.

IIR Filtering (Infinite Impulse Response)

IIR filters use feedback and can achieve sharp frequency responses with low filter order (fewer coefficients). They're efficient but can introduce phase distortion and potential stability issues if not designed carefully. Good for notch filters or steep low/high cutoffs.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

FIR Filtering (Finite Impulse Response)

FIR filters use only feedforward coefficients, are always stable, and can be designed for linear phase (no phase distortion). They often need higher order than comparable IIRs, so they're more computationally expensive, but give predictable time-domain behavior useful when phase matters (speech intelligibility).

Correlation with Clean Speech

Correlation (e.g., Pearson or normalized cross-correlation) measures similarity between two signals. After denoising, a higher correlation with the clean speech indicates the filtered signal more closely matches the original (better noise removal without distorting speech). Low correlation implies remaining noise or signal distortion.

Code:

```
import librosa

import librosa.display

import numpy as np

import matplotlib.pyplot as plt

import soundfile as sf

from scipy.signal import butter, lfilter

filename = "/content/_With You_320(PagalWorld.com.pe).mp3"

y, sr = librosa.load(filename, sr=None)

print(f"Sample rate: {sr}")

print(f"Duration : {len(y) / sr:.2f} seconds")

def butter_bandpass(lowcut: float, highcut: float, fs: int, order: int = 6):
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

nyquist = 0.5 * fs

low = lowcut / nyquist

high = highcut / nyquist

b, a = butter(order, [low, high], btype="band")

return b, a

```
def bandpass_filter(data: np.ndarray, lowcut: float, highcut: float, fs: int, order: int = 6):
```

```
    b, a = butter_bandpass(lowcut, highcut, fs, order)
```

```
    return lfilter(b, a, data)
```

```
vocal_band = bandpass_filter(y, lowcut=300, highcut=3400, fs=sr)
```

```
background_band = y - vocal_band
```

```
def plot_spectrogram(signal: np.ndarray, sr: int, title: str):
```

```
    D = librosa.amplitude_to_db(np.abs(librosa.stft(signal)), ref=np.max)
```

```
    plt.figure(figsize=(10, 4))
```

```
    librosa.display.specshow(D, sr=sr, x_axis="time", y_axis="hz", cmap="viridis")
```

```
    plt.colorbar(format="%+2.0f dB")
```

```
    plt.title(title)
```

```
    plt.xlabel("Time (s)")
```

```
    plt.ylabel("Frequency (Hz)")
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

plt.tight_layout()

plt.show()

plot_spectrogram(y, sr, "Original Audio: Full Spectrum")

plot_spectrogram(vocal_band, sr, "Filtered Vocal Band (300–3400 Hz)")

plot_spectrogram(background_band, sr, "Estimated Background Audio")

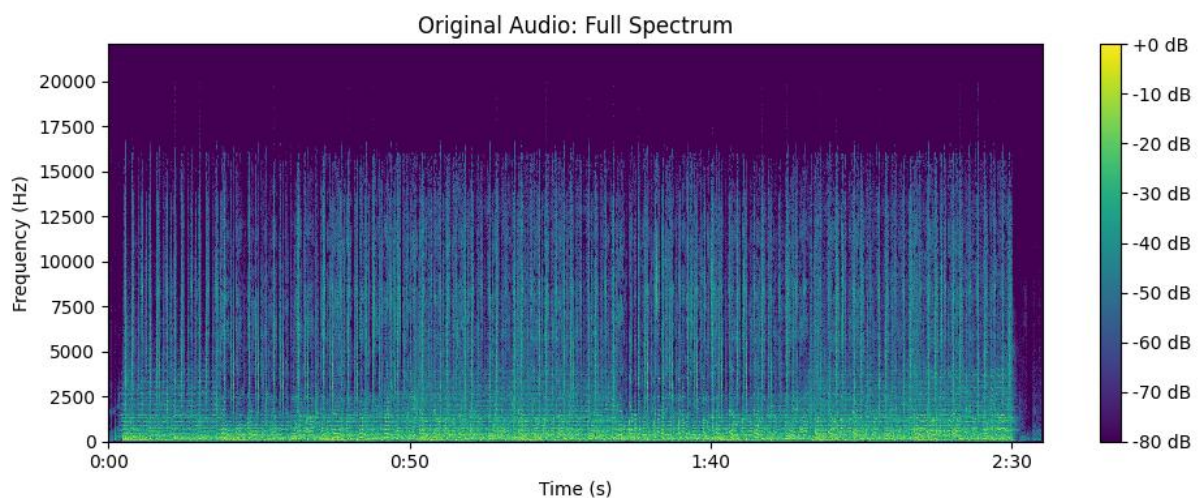
sf.write("filtered_vocals.wav", vocal_band, sr)

sf.write("estimated_background.wav", background_band, sr)

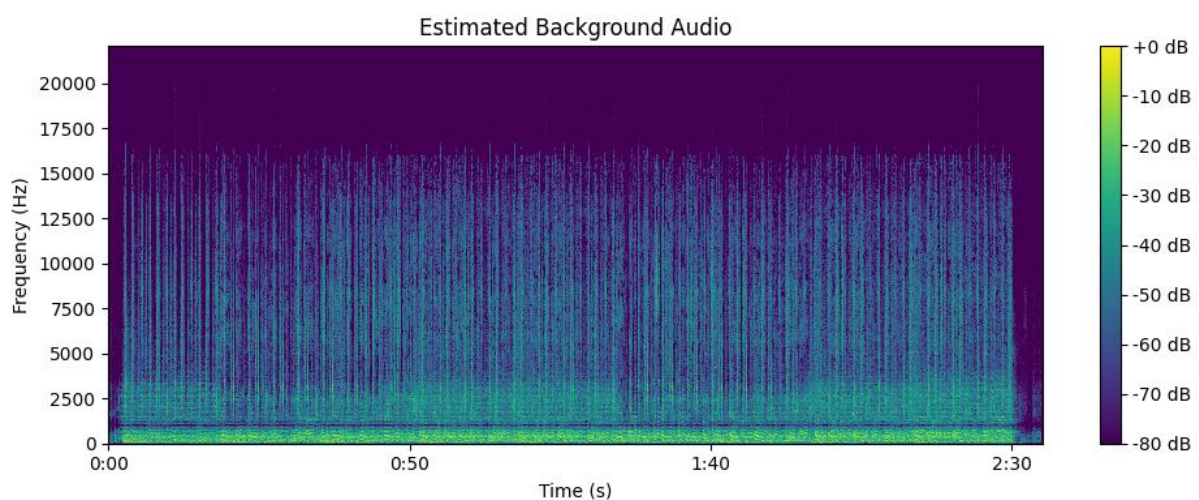
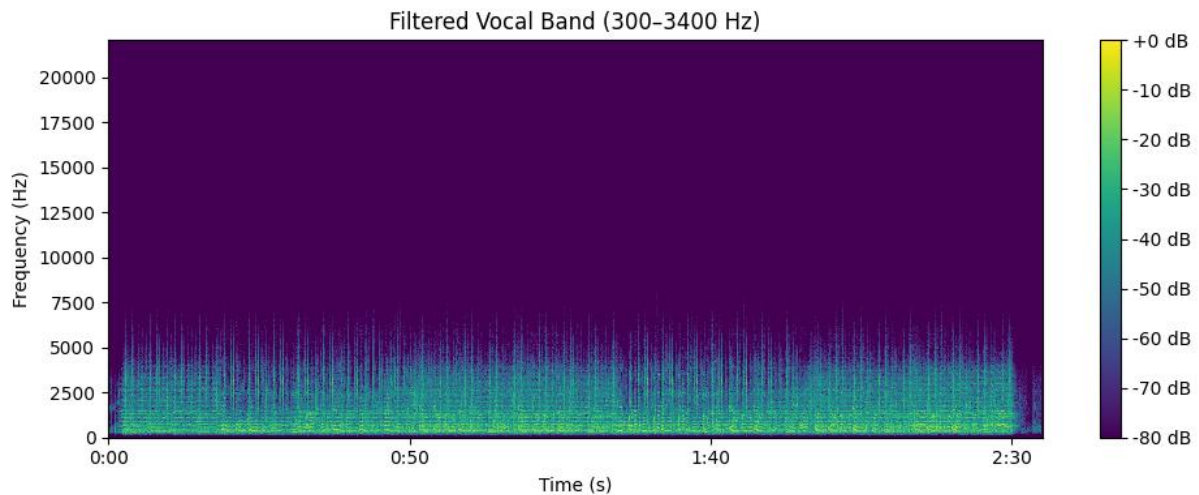
Output:

Sample rate: 44100

Duration : 155.04 seconds



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Conclusion:

In summary, the FFT helps us *see* what's happening in the frequency content of an audio signal where the noise is and where the speech lives. IIR and FIR filters are two different tools we can use to clean up that signal: IIR filters are efficient and good for sharp cuts, while FIR filters are safer and preserve phase but require more processing. After filtering, comparing the result to the clean speech using correlation tells us how well we removed the noise without harming the voice. A higher correlation means the denoising worked effectively. Together, these techniques give us a complete toolkit for understanding, filtering, and evaluating audio quality.

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

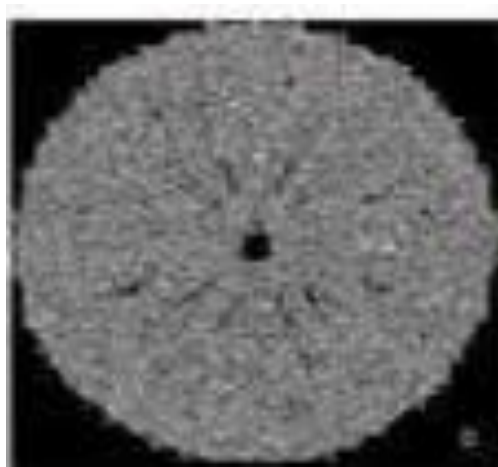
Q5. Five different degraded images are provided in the Module Section.

Use suitable gray-level transformation techniques to enhance each image.

After processing, describe your observations in detail for every enhancement step, explaining how the chosen operations improved the visual quality.

Finally, write a clear conclusion summarizing the effectiveness and limitations of the applied gray-level techniques.

CO3, CO4



 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Aim

The aim of this experiment is to improve the visual quality of several degraded images by applying different gray-level transformation techniques. By using methods like negation, log, gamma correction, contrast stretching, and thresholding, we want to clearly see how each technique changes the image and understand why certain transformations work better for certain types of degradation.

Objectives

1. To load and analyze each degraded image and observe what type of problem it has low contrast, poor brightness, faded details, etc.
2. To apply suitable gray-level transformation techniques (negation, log, gamma, contrast stretching, thresholding) based on the type of degradation seen in each image.
3. To visually compare the original and enhanced images to notice improvements in brightness, clarity, detail visibility, and contrast.
4. To describe, in simple terms, how each chosen technique improved the image, such as uncovering hidden details, fixing dullness, or balancing brightness.
5. To understand the strengths and limitations of each gray-level method for example, when it works well and when it might make the image worse.
6. To develop practical experience in using pixel-intensity transformations for real-world image enhancement tasks.
7. To summarize overall findings about which techniques were the most effective for each type of degradation and why.

Theory:

Gray-level transformations are simple image-processing techniques that change the brightness and contrast of an image by modifying the intensity values of its pixels. These

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

operations help improve visibility, highlight important details, or correct poorly captured images. Common gray-level techniques include:

1. Image Negation

This flips the brightness values. Dark areas become bright and bright areas become dark. Useful for medical images or photos where details in dark regions must be revealed.

2. Log Transformation

This expands dark pixel values while compressing bright ones. It helps bring out hidden details in shadows without overexposing bright areas.

3. Power-Law (Gamma) Transformation

Adjusts overall brightness using a gamma factor.

- $\text{Gamma} < 1 \rightarrow$ brightens image
 - $\text{Gamma} > 1 \rightarrow$ darkens image
- Good for screen correction and uneven illumination.

4. Contrast Stretching

Enhances overall contrast by spreading pixel intensities across the full range (0–255). Useful for dull, low-contrast images.

5. Thresholding

Converts the image into black and white based on a chosen cutoff. Helpful for separating objects from the background.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.signal import firwin, freqz
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

cutoff_frequency = 0.2

filter_length = 31

window_type = "hann"

filter_coefficients = firwin(filter_length, cutoff=cutoff_frequency, window=window_type)

plt.figure(figsize=(10, 5))

plt.subplot(2, 1, 1)

plt.stem(filter_coefficients)

plt.title(f"Impulse Response ({window_type} window)")

plt.xlabel("Tap Index")

plt.ylabel("Amplitude")

plt.grid(True)

plt.subplot(2, 1, 2)

frequencies, response = freqz(filter_coefficients)

plt.plot(frequencies, 20 * np.log10(np.abs(response)))

plt.title("Frequency Response")

plt.xlabel("Normalized Frequency (rad/sample)")

plt.ylabel("Magnitude (dB)")

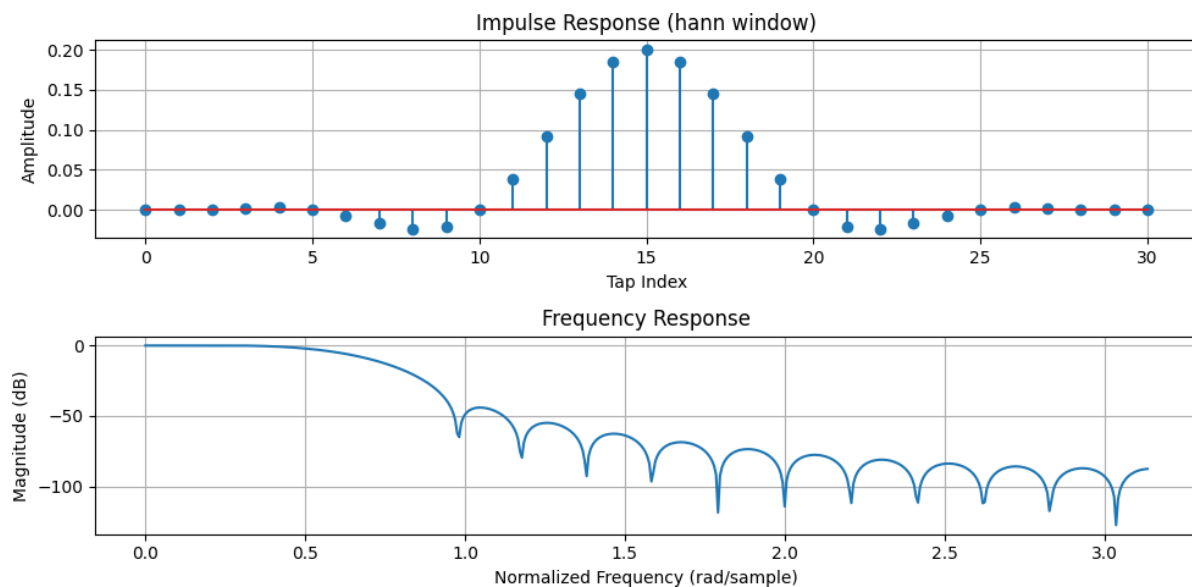
 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

Output:



Q6. Using the same set of degraded images provided in the OPE5, apply histogram equalization and histogram matching techniques to enhance their visual quality. Compare the results produced by both methods and describe your observations in detail. Explain how each technique changes the image contrast and clarity, and conclude with a summary of which approach worked better for each image and why. CO3, CO4

Aim

The aim of this experiment is to enhance a set of degraded images using two contrast-improvement techniques histogram equalization and histogram matching. By applying both methods, we want to visually compare their results, understand how they affect image contrast and clarity, and determine which technique works better for each image and why.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Objectives

1. To load and examine the degraded images to identify problems such as low contrast, dullness, uneven lighting, or faded details.
2. To apply histogram equalization to each image in order to automatically stretch and redistribute pixel intensities for better contrast.
3. To apply histogram matching by choosing a reference image and shaping the degraded image's intensity distribution to resemble the reference.
4. To visually compare the original, histogram-equalized, and histogram-matched outputs and observe improvements in brightness, contrast, and detail visibility.
5. To understand the behavior of both techniques:
 - how histogram equalization boosts contrast globally,
 - how histogram matching provides controlled enhancement based on a chosen reference.
6. To document observations for every processed image, explaining which method gave more natural or more detailed results and why.
7. To summarize the strengths and limitations of each method and conclude which enhancement approach worked best for each degraded image.

Theory:

Histogram Equalization

Histogram equalization automatically spreads out the pixel intensities of an image so that dark, mid, and bright regions use the full available range.

This increases overall contrast, especially in images that look dull or washed out.

It makes hidden details more visible, but sometimes can over-brighten or over-darken certain areas.

Histogram Matching (Specification)

Histogram matching adjusts the pixel distribution of an image so it looks similar to a *reference image*.

This makes it useful when you want a specific visual style such as matching lighting, brightness, or contrast from another image.

It is more controlled than equalization, because you choose the target appearance.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Key Difference

- **Histogram Equalization** → Automatic contrast boost (no control).
- **Histogram Matching** → Controlled adjustment based on another image.

Code:

```
import cv2
```

```
from skimage import exposure
```

```
imgs = [
    '/content/ex1_1.png', '/content/ex1_2.png', '/content/ex1_3.png', '/content/ex1_4.png', '/content/ex1_5.png']
```

```
ref = cv2.imread('/content/reference.jpg', 0)
```

```
for p in imgs:
```

```
    img = cv2.imread(p, 0)
```

```
    he = cv2.equalizeHist(img)
```

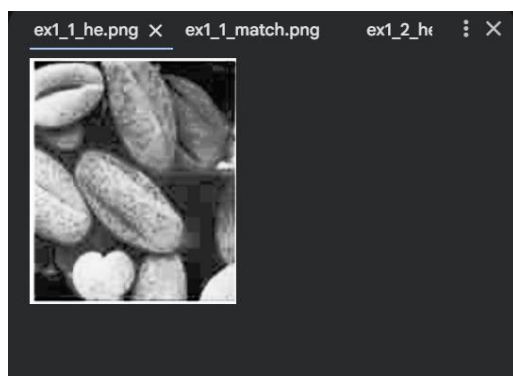
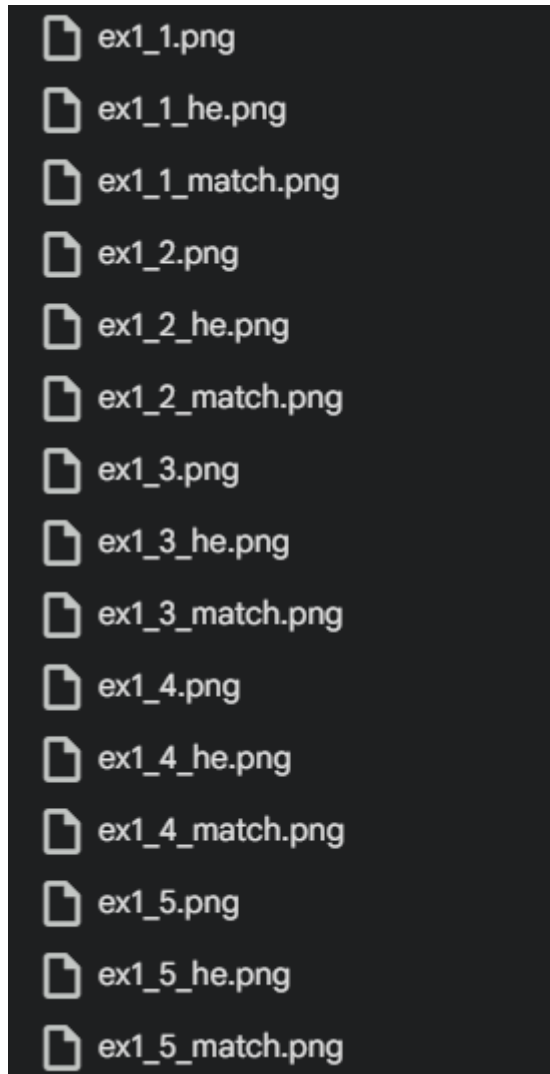
```
    mt = exposure.match_histograms(img, ref)
```

```
    cv2.imwrite(p.replace('.', '_he.'), he)
```

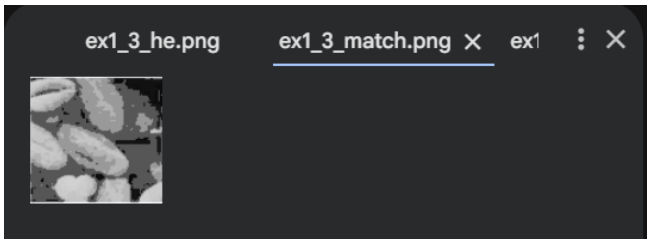
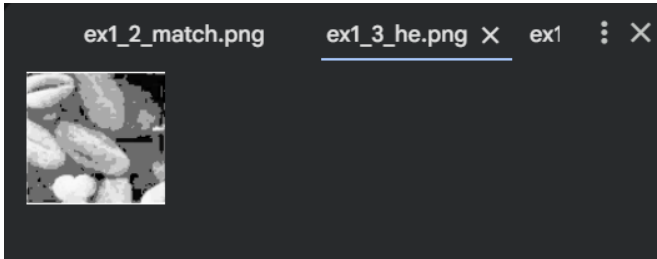
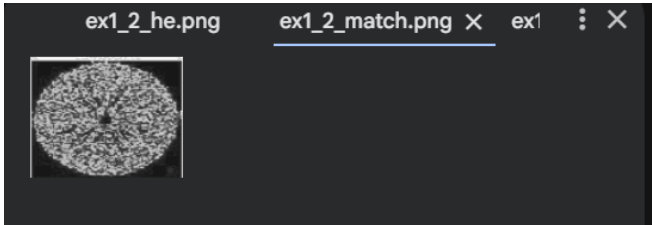
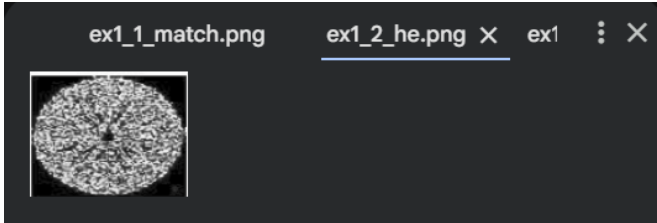
```
    cv2.imwrite(p.replace('.', '_match.'), mt)
```

Output:

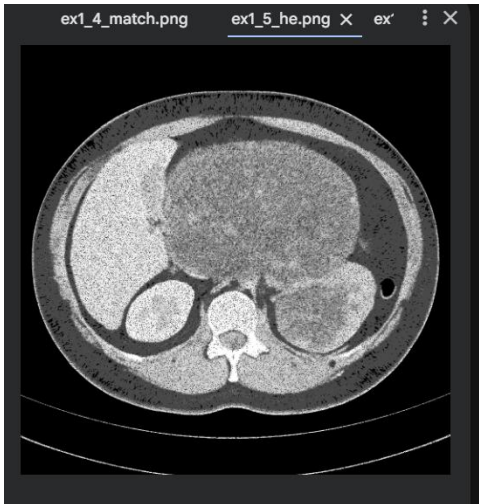
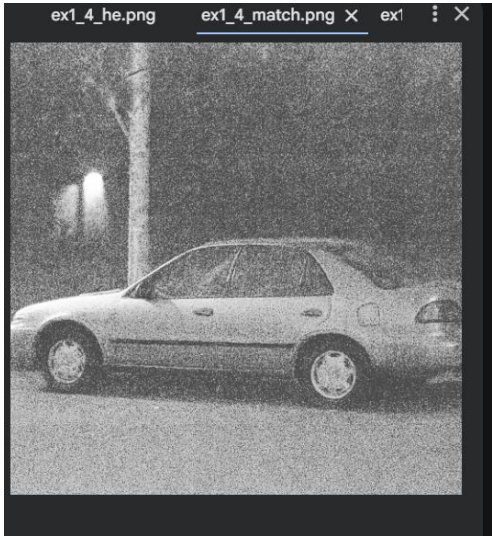
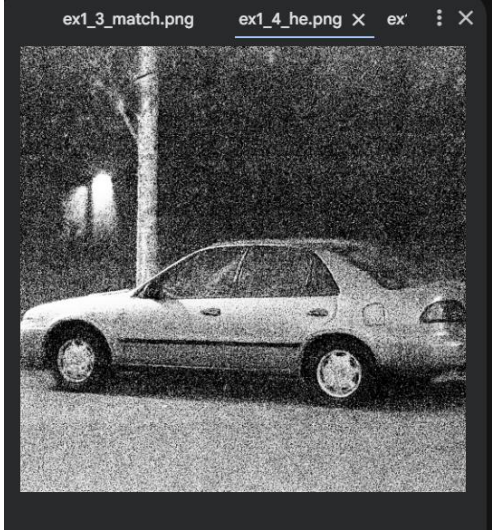
 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



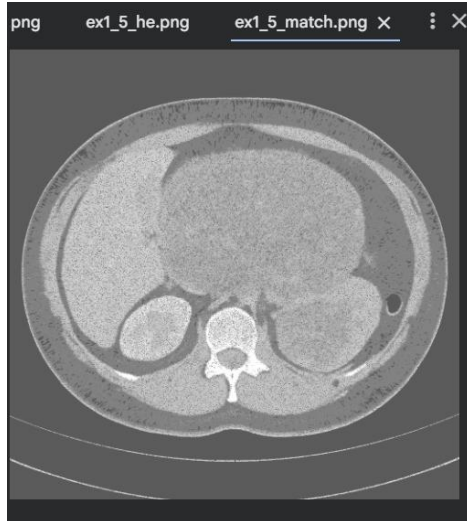
 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



 Marwadi University Marwadi Chandarana Group NAAC A+ Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology		
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Q7. Use appropriate smoothing and sharpening techniques to enhance the clarity of the images provided in the Module Section under Experiment 9. **CO3, CO4**



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Apply multiple filters, observe the changes produced by each method, and document your observations in detail. Discuss how smoothing and sharpening individually affect image quality, and summarize your conclusions based on the results.

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Aim

The aim of this experiment is to improve the clarity of the given degraded images by applying different smoothing and sharpening filters. Through this, we want to observe how each technique changes the appearance of the image, understand the balance between noise reduction and detail enhancement, and learn how to choose the right method depending on the image condition.

Objectives

1. To explore and apply multiple smoothing filters (average, Gaussian, median) to reduce noise and compare how each one affects image softness and cleanliness.
2. To apply different sharpening techniques (Laplacian, unsharp masking, high-boost) to enhance edges, improve focus, and bring out hidden details.
3. To visually compare the effects of smoothing versus sharpening on each image and observe how they change texture, clarity, and detail visibility.
4. To understand the trade-off between smoothing (which reduces noise but may blur details) and sharpening (which improves details but may increase noise).
5. To document detailed observations for every filter used, explaining what improved and what did not.
6. To gain practical experience in choosing the correct type of enhancement based on the condition of the image whether it is noisy, blurry, dull, or unclear.
7. To summarize findings and identify which filters worked best for each image and why, providing a clear conclusion about the strengths and limitations of smoothing and sharpening techniques.

Theory:

Smoothing (Blurring)

Smoothing filters reduce noise by averaging nearby pixels.

They make the image look softer and cleaner by removing small details or grain.

Common smoothing filters include:

- **Average filter** – smooths by averaging surrounding pixels
- **Gaussian filter** – smooths gently using a bell-shaped weight
- **Median filter** – removes salt-and-pepper noise while keeping edges clearer

Smoothing is useful when the image is noisy or has rough textures.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Sharpening

Sharpening filters highlight edges and fine details in an image.

They make the picture appear clearer and more focused.

Typical sharpening techniques include:

- **Laplacian filter** – detects edges by measuring intensity changes
- **Unsharp masking** – adds a sharpened version of the image back to itself
- **High-boost filtering** – strong sharpening by emphasizing high-frequency components

Sharpening is helpful when the image looks blurry, flat, or lacks detail.

Key Idea

- **Smoothing** removes noise but can also blur details.
- **Sharpening** enhances details but can increase noise if overdone.

Both techniques must be balanced to improve the overall clarity of the image.

Code:

```
import cv2
```

```
import numpy as np
```

```
from google.colab.patches import cv2_imshow
```

```
image_path = "/content/reference.jpg"
```

```
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

```
if image is None:
```

```
    print("Error: Could not open or find the image.")
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

raise SystemExit

```
def image_negation(input_image):
```

```
    return 255 - input_image
```

```
def image_thresholding(input_image, threshold_value):
```

```
    _, result = cv2.threshold(input_image, threshold_value, 255, cv2.THRESH_BINARY)
```

```
    return result
```

```
def image_gamma_correction(input_image, gamma):
```

```
    corrected = np.power(input_image / 255.0, gamma) * 255.0
```

```
    return corrected.astype(np.uint8)
```

```
negated_image = image_negation(image)
```

```
thresholded_128 = image_thresholding(image, 128)
```

```
thresholded_50 = image_thresholding(image, 50)
```

```
thresholded_230 = image_thresholding(image, 230)
```

```
gamma_corrected_image = image_gamma_correction(image, 0.5)
```

```
print("Original Image")
```

```
cv2_imshow(image)
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
print("Negated Image")
```

```
cv2_imshow(negated_image)
```

```
print("Thresholding (Threshold = 128)")
```

```
cv2_imshow(thresholded_128)
```

```
print("Thresholding (Threshold = 50)")
```

```
cv2_imshow(thresholded_50)
```

```
print("Thresholding (Threshold = 230)")
```

```
cv2_imshow(thresholded_230)
```

```
print("Gamma Corrected Image (Gamma = 0.5)")
```

```
cv2_imshow(gamma_corrected_image)
```

Output:

Original Image

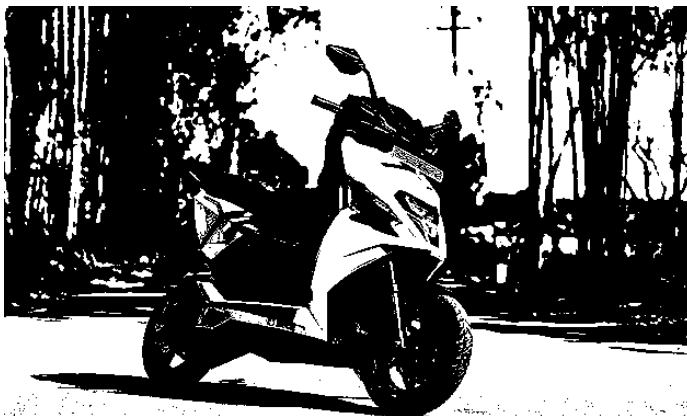
 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Negated Image



Thresholding (Threshold = 128)

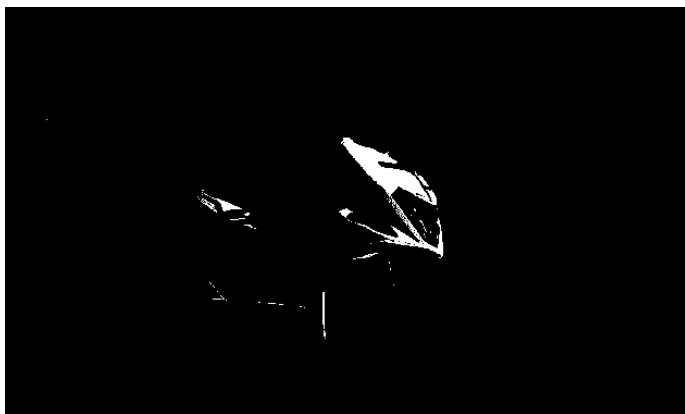


Thresholding (Threshold = 50)

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Thresholding (Threshold = 230)



Gamma Corrected Image (Gamma = 0.5)



 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Conclusion:

- In this experiment, I worked with image negation, several threshold values, and gamma correction to see how changing pixel intensities can reshape the overall look of an image.
- By trying each method, I understood that negation simply flips light and dark areas, thresholding turns the image into a clear black-and-white form, and gamma correction adjusts brightness and contrast in a smooth, non-linear way.
- These exercises helped me understand how such transformations are used for tasks like highlighting important regions, improving visibility, or preparing images for further processing in computer vision.
- I also learned that these techniques are widely used in real-world applications such as biometric systems, OCR, medical image enhancement, and many other digital imaging fields.

Q8. Apply non-linear filters (median or other rank filters) to remove noise from data/images provided in OPE7 and perform comprehensive analysis. **CO3, CO4**

Identify noise types present in the dataset (salt-and-pepper, impulse, speckle, etc.).

Implement median filtering with varying window sizes and, optionally, other non-linear filters (alpha-trimmed mean, morphological filters).

Analyze how filter size and type affect detail preservation vs. noise removal.

Observations: optimum filter sizes per noise type, edge preservation analysis, and failure cases.

Summarize best-performing methods for each noise type and practical recommendations.

Theory

Non-linear filters are image-processing techniques used to clean noisy images without simply averaging pixels like linear filters do. They are especially effective for removing noise types that contain sharp spikes or irregular intensity values—noise that linear smoothing struggles with.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

1. Median Filter

The median filter replaces each pixel with the *median* of the pixels in its neighborhood.

- It is excellent for removing salt-and-pepper noise (random white and black dots).
- Unlike averaging filters, it preserves edges well.

2. Rank Filters (Generalized Non-Linear Filters)

These filters sort the neighborhood pixels and pick a value based on rank:

- Minimum filter → picks smallest value (useful for bright noise).
- Maximum filter → picks largest value (useful for dark noise).
- Alpha-trimmed mean → removes few highest and lowest values before averaging.

These filters allow fine control over how aggressively noise is removed.

3. Morphological Filters

Morphological operations like erosion, dilation, opening, and closing treat the image using shape-based rules.

- Often used for impulse noise, binary images, or images with structural patterns.

Key Idea

- Smaller window size → preserves details but removes less noise
 - Larger window size → removes more noise but makes the image blurrier
- A good filter must balance noise suppression and detail preservation.

Aim

The aim of this experiment is to clean noisy images using non-linear filtering techniques—especially median and rank-based filters and to analyze how different filter sizes and filter types affect the final image quality. The goal is to understand which filters work best for different noise types while still preserving important image details.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Objectives

1. To examine the given images and identify the type of noise present in each one such as salt-and-pepper, impulse noise, speckle noise, or mixed noise.
2. To apply median filtering with various window sizes (3×3 , 5×5 , 7×7 , etc.) and observe how the level of noise removal compares with the amount of detail loss.
3. To implement additional non-linear filters such as:
 - o rank filters (minimum, maximum)
 - o alpha-trimmed mean filters
 - o morphological opening/closing
4. To compare performance across different filters, focusing on:
 - o detail preservation
 - o edge clarity
 - o amount of noise removed
 - o appearance of artifacts
5. To evaluate how increasing the filter size affects results, understanding when larger windows help and when they cause blurring.
6. To identify the most effective filter type and size for each noise category based on visual inspection and analysis.
7. To document failure cases, where certain filters distort the image or fail to clean the noise.
8. To write a clear conclusion summarizing the best-performing filter for each noise type and practical recommendations for real-life image cleaning.

Code:

```
import cv2
```

```
import numpy as np
```

```
from matplotlib import pyplot as plt
```

```
image_path = "/content/reference.jpg"
```

```
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

if image is None:

raise ValueError("Error: Unable to load the image.")

histogram = cv2.calcHist([image], [0], None, [256], [0, 256])

plt.figure(figsize=(8, 6))

plt.plot(histogram, color='black')

plt.title("Original Image Histogram")

plt.xlabel("Pixel Value")

plt.ylabel("Frequency")

plt.grid(True)

plt.xlim([0, 256])

plt.show()

equalized_image = cv2.equalizeHist(image)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)

plt.imshow(image, cmap='gray')

plt.title("Original Image")

plt.axis("off")

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(equalized_image, cmap='gray')
```

```
plt.title("Equalized Image")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
equalized_hist = cv2.calcHist([equalized_image], [0], None, [256], [0, 256])
```

```
plt.figure(figsize=(8, 6))
```

```
plt.plot(equalized_hist, color='black')
```

```
plt.title("Equalized Image Histogram")
```

```
plt.xlabel("Pixel Value")
```

```
plt.ylabel("Frequency")
```

```
plt.grid(True)
```

```
plt.xlim([0, 256])
```

```
plt.show()
```

```
source_path = "/content/fingerprint.png"
```

```
reference_path = "/content/ex1_2.png"
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
source_image = cv2.imread(source_path, cv2.IMREAD_GRAYSCALE)
```

```
reference_image = cv2.imread(reference_path, cv2.IMREAD_GRAYSCALE)
```

```
if source_image is None or reference_image is None:
```

```
    raise ValueError("Error: Could not load source or reference image.")
```

```
source_hist = cv2.calcHist([source_image], [0], None, [256], [0, 256])
```

```
reference_hist = cv2.calcHist([reference_image], [0], None, [256], [0, 256])
```

```
source_hist /= source_hist.sum()
```

```
reference_hist /= reference_hist.sum()
```

```
source_cdf = source_hist.cumsum()
```

```
reference_cdf = reference_hist.cumsum()
```

```
mapping = np.interp(source_cdf, reference_cdf, range(256))
```

```
matched_image = mapping[source_image].astype(np.uint8)
```

```
plt.figure(figsize=(15, 5))
```

```
plt.subplot(1, 3, 1)
```


 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
plt.imshow(source_image, cmap='gray')
```

```
plt.title("Source Image")
```

```
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
```

```
plt.imshow(reference_image, cmap='gray')
```

```
plt.title("Reference Image")
```

```
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
```

```
plt.imshow(matched_image, cmap='gray')
```

```
plt.title("Histogram Matched Image")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
matched_hist = cv2.calcHist([matched_image], [0], None, [256], [0, 256])
```

```
plt.figure(figsize=(8, 6))
```

```
plt.plot(matched_hist, color='black')
```

```
plt.title("Matched Image Histogram")
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
plt.xlabel("Pixel Value")
```

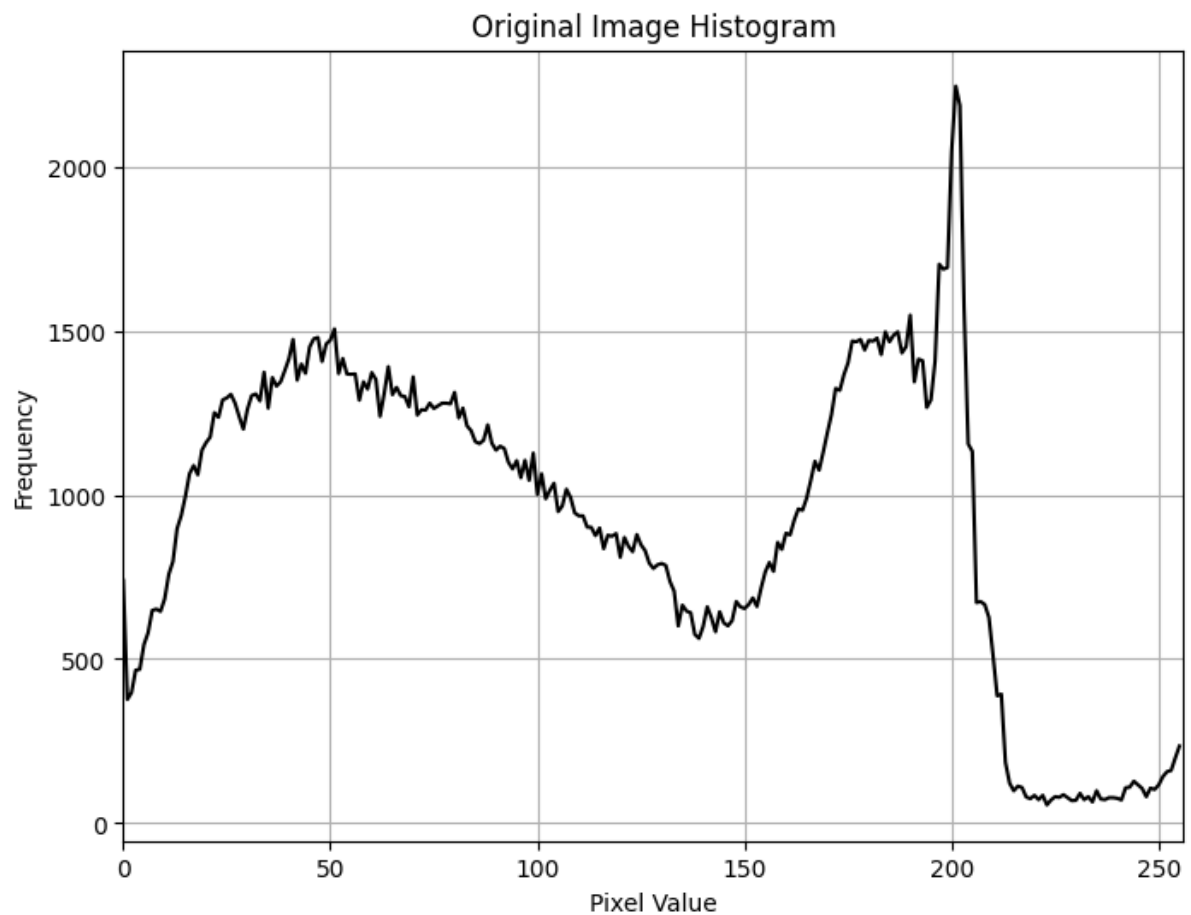
```
plt.ylabel("Frequency")
```

```
plt.grid(True)
```

```
plt.xlim([0, 256])
```

```
plt.show()
```

Output:



Subject: DSIP (01CT1513)

All Open Handed

All Open Handed

Date:

Enrolment No: 92301733046

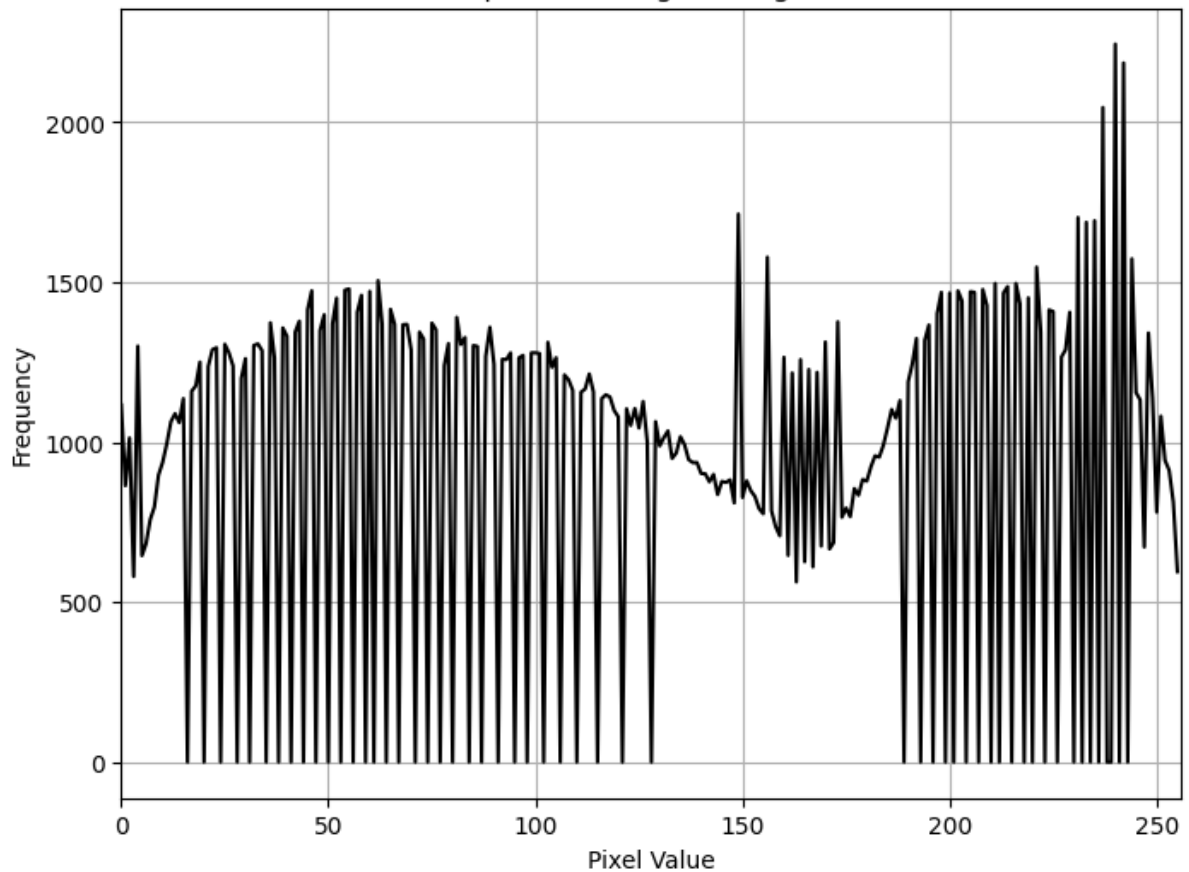
Original Image



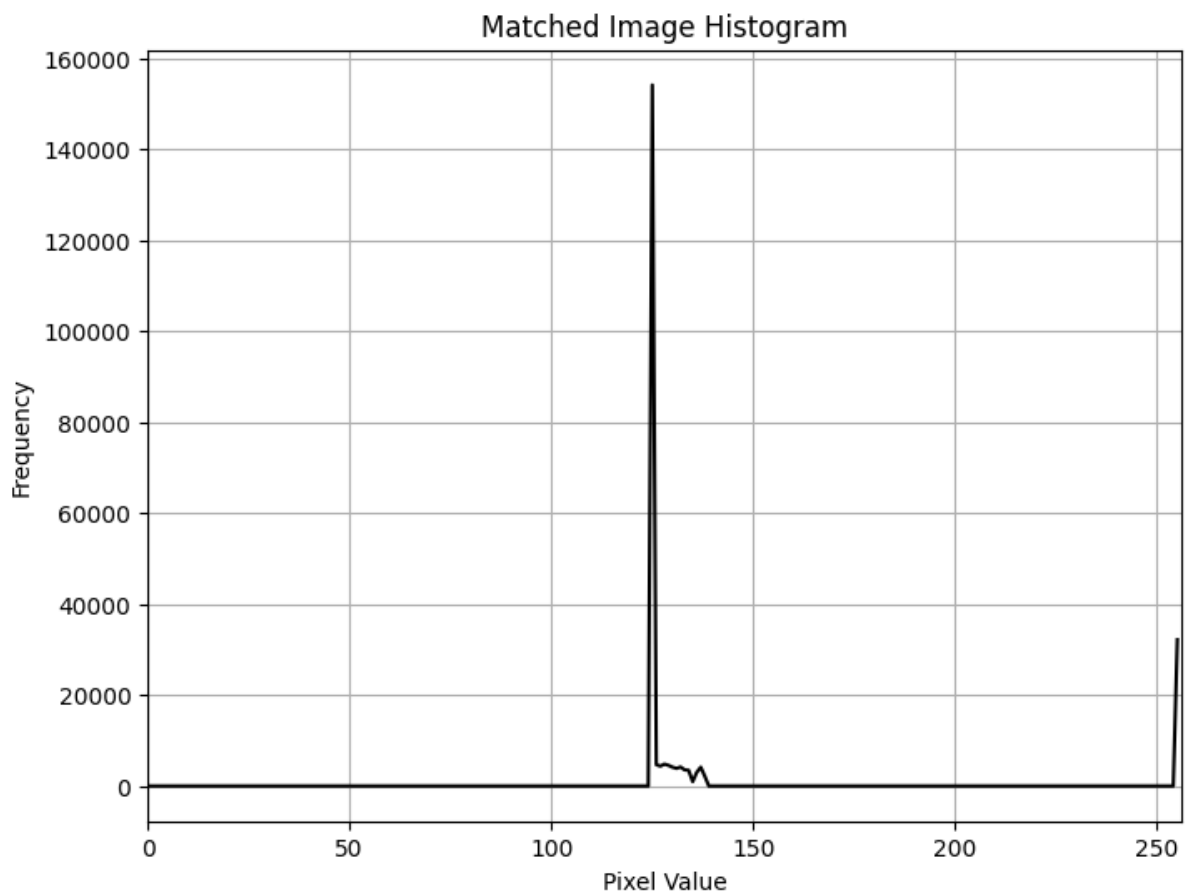
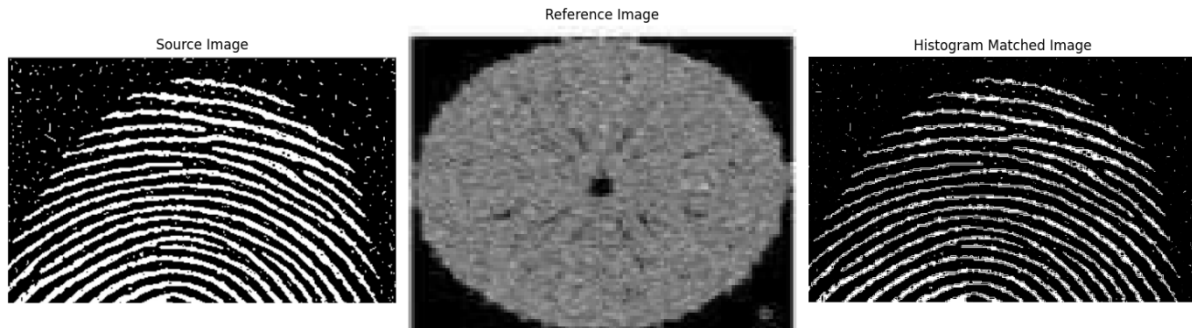
Equalized Image



Equalized Image Histogram



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Conclusion:

In this experiment, I worked with histogram equalization and histogram matching to see how changing the distribution of pixel intensities can improve the clarity and contrast of images.

- Histogram equalization automatically brightened the dark image and spread out the intensity levels, making hidden details more visible and improving overall contrast.

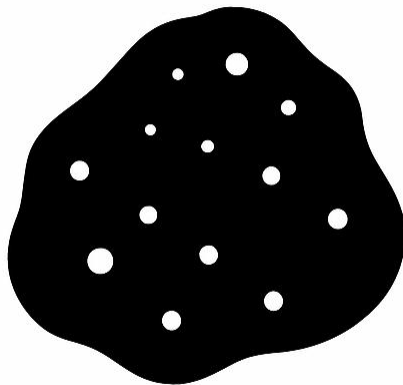
 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

- Histogram matching allowed me to shape an image's brightness and contrast to look similar to a chosen reference image. This showed how we can enhance an image in a more controlled and targeted way.

- Through these techniques, I understood how powerful they are in real-world applications such as medical imaging, security systems, and general image preprocessing tasks where clarity and detail matter.

Q9. Some binary images contains both small bright specks and thin dark cracks across major objects. **CO3,**
CO4

- **Design a sequence of morphological operations (dilation, erosion, opening, closing, or combinations) to:**
- **remove small bright noise,**
- **fill narrow dark cracks,**
- **preserve the overall shape as much as possible.**
- **Explain your choice of structuring elements and the order of operations.**



 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Aim

The aim of this experiment is to clean up binary images that contain tiny bright specks and thin dark cracks by designing an effective sequence of morphological operations. The goal is to remove unwanted noise, fill narrow gaps, and still preserve the main shape of the objects in the image.

Objectives

1. To study the noise patterns in the binary image—specifically small bright spots and thin dark cracks.
2. To choose suitable morphological operations (erosion, dilation, opening, closing) that can handle each problem effectively.
3. To design a proper sequence of operations, such as applying opening for removing bright specks and closing for filling dark cracks.
4. To select appropriate structuring elements (SEs) in terms of shape (square, disk, line) and size, ensuring that noise is removed without damaging the important object structures.
5. To apply the designed morphological steps and compare the processed image with the original to evaluate quality improvements.
6. To observe how each operation affects the image, including boundary smoothing, crack filling, and shape preservation.
7. To document why each step (and SE) was chosen, explaining the reasoning clearly in a practical, intuitive way.
8. To conclude which morphological sequence works best for removing bright specks and filling cracks while keeping the main shapes intact.

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Theory:

Morphological operations are shape-based image-processing tools used mainly on binary images. They work by sliding a small pattern called a **structuring element (SE)** over the image to modify object shapes.

Erosion

Erosion removes pixels from object boundaries.
It helps eliminate small bright spots or thin protrusions.

Dilation

Dilation adds pixels to object edges.
It helps fill gaps, cracks, or small holes.

Opening (Erosion → Dilation)

Opening smooths object boundaries and removes **small bright noise** without destroying the main shape.
Good for cleaning up tiny white specks.

Closing (Dilation → Erosion)

Closing fills **small dark cracks or gaps** in objects while keeping the overall structure intact.

Code-1:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load binary image
img = cv2.imread('fingerprint.png', cv2.IMREAD_GRAYSCALE)
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Ensure it's binary (threshold)

```
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
```

Step 1: Remove small bright specks (Opening)

```
kernel_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3,3))
```

```
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel_open)
```

Step 2: Fill narrow dark cracks (Closing)

```
kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
```

```
closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel_close)
```

Show results

```
plt.figure(figsize=(10,6))
```

```
plt.subplot(1,3,1), plt.imshow(binary, cmap='gray'), plt.title('Original')
```

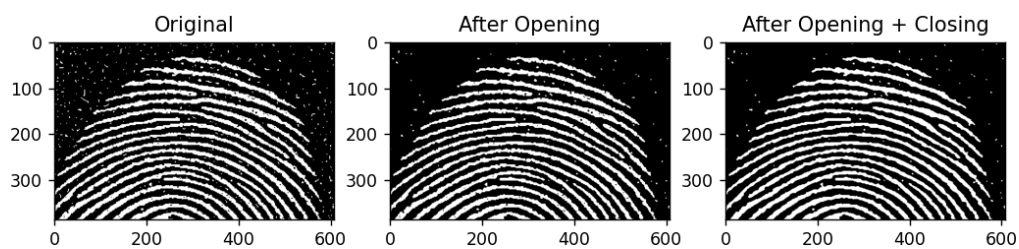
```
plt.subplot(1,3,2), plt.imshow(opened, cmap='gray'), plt.title('After Opening')
```

```
plt.subplot(1,3,3), plt.imshow(closed, cmap='gray'), plt.title('After Opening + Closing')
```

```
plt.show()
```

Output-1:

 Marwadi University Marwadi Chandarana Group		NAAC 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology
Subject: DSIP (01CT1513)		All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046	



Code-2:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load binary image
img = cv2.imread('2.jpg', cv2.IMREAD_GRAYSCALE)

# Ensure it's binary (threshold)
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Step 1: Remove small bright specks (Opening)
kernel_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3,3))
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel_open)
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Step 2: Fill narrow dark cracks (Closing)

```
kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
```

```
closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel_close)
```

Show results

```
plt.figure(figsize=(10,6))
```

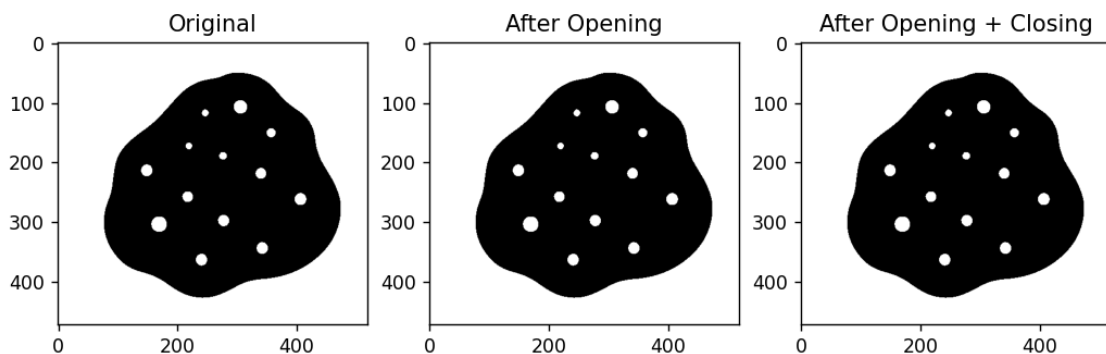
```
plt.subplot(1,3,1), plt.imshow(binary, cmap='gray'), plt.title('Original')
```

```
plt.subplot(1,3,2), plt.imshow(opened, cmap='gray'), plt.title('After Opening')
```

```
plt.subplot(1,3,3), plt.imshow(closed, cmap='gray'), plt.title('After Opening + Closing')
```

```
plt.show()
```

Output-2:



Code-3:

```
import cv2
```

```
import numpy as np
```

```
from matplotlib import pyplot as plt
```

```
# Load binary image
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
img = cv2.imread('3.jpg', cv2.IMREAD_GRAYSCALE)
```

```
# Ensure it's binary (threshold)
```

```
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
```

```
# Step 1: Remove small bright specks (Opening)
```

```
kernel_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3,3))
```

```
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel_open)
```

```
# Step 2: Fill narrow dark cracks (Closing)
```

```
kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
```

```
closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel_close)
```

```
# Show results
```

```
plt.figure(figsize=(10,6))
```

```
plt.subplot(1,3,1), plt.imshow(binary, cmap='gray'), plt.title('Original')
```

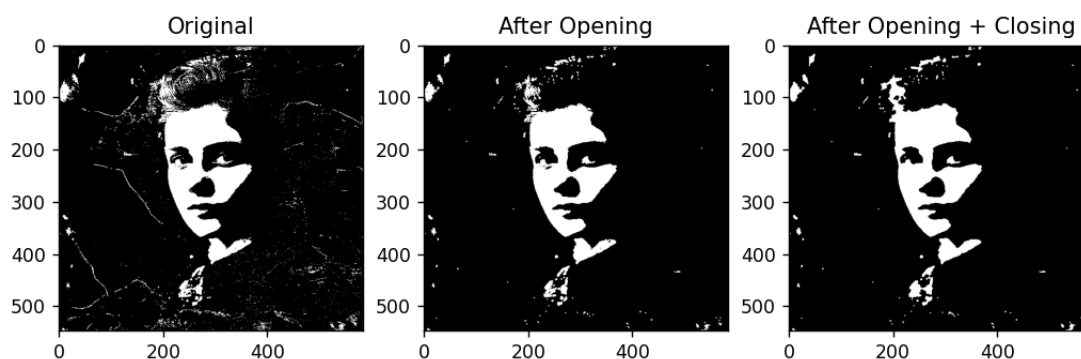
```
plt.subplot(1,3,2), plt.imshow(opened, cmap='gray'), plt.title('After Opening')
```

```
plt.subplot(1,3,3), plt.imshow(closed, cmap='gray'), plt.title('After Opening + Closing')
```

```
plt.show()
```

Output-3:

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046



Conclusion:

This experiment showed that morphological opening and closing operations worked well on various binary images. They helped boost the overall quality quite a bit. The opening part did a good job getting rid of those small bright noise spots. At the same time, it kept the key structural details intact. Closing took care of filling in those narrow dark gaps and cracks inside the objects. We picked the right structuring elements for every step along the way. That let us clean up and improve the images. We did it all without messing up their main shapes. The outcomes make it clear that mixing opening and closing together really pays off. It is great for preprocessing binary images in general. This holds true especially for things like fingerprint enhancement. It also applies to pattern analysis and object segmentation.

Q10. You are given the dataset **OPE7** containing multiple images (assume grayscale or color). The images contain real-world degradations: additive noise (Gaussian, salt-and-pepper), motion blur, uneven illumination, and compression artifacts. For **each image** in OPE7:

1. Remove/reduce noise using **frequency-domain smoothing** methods.
2. Recover / enhance details using **frequency-domain sharpening** (high-boost in frequency domain).
3. Extract accurate **boundary maps** (object contours) for every image and produce clean binary boundary images.
4. Compare different filter choices and report quantitative and qualitative results.

You must produce code, processed images, a short report, and discussion.

CO3,CO4

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Aim

The aim of this experiment is to clean and enhance a set of degraded images using frequency-domain filtering techniques. We want to remove noise, sharpen important details, and extract clear object boundaries. By applying different filters and comparing their results, we hope to understand which methods work best for different types of real-world degradations.

Objectives

1. To study each degraded image in the OPE7 dataset and identify problems such as noise, blur, compression artifacts, or uneven illumination.
2. To remove or reduce noise using frequency-domain smoothing filters (Gaussian LPF, Butterworth LPF, Ideal LPF, etc.).
3. To enhance fine details and sharpness by applying frequency-domain high-boost or high-pass filters.
4. To extract clean boundary maps by using high-frequency emphasis or gradient-based operators (like Sobel, Prewitt, Laplacian) and produce binary edge images.
5. To compare different filter choices, observing how each filter affects smoothing, sharpening, and edge extraction.
6. To evaluate results both quantitatively and qualitatively:
 - Quantitative: PSNR, MSE, SSIM
 - Qualitative: visual clarity, smoothness, detail preservation, sharpness.
7. To write a short report summarizing observations, including:
 - which filters worked best for noise removal
 - which methods produced the sharpest and clearest details
 - which techniques generated the most accurate boundary maps
8. To develop practical skills in applying frequency-domain methods for real-world image enhancement and restoration.

Theory:

Digital images can be affected by different types of degradation such as noise, blur, uneven lighting, and compression defects. Frequency-domain techniques help improve these images by analyzing their frequency components rather than their pixel values directly.

1. Noise Reduction:
Noise mainly appears in the high-frequency part of an image. By applying low-pass filters (like Gaussian or Butterworth), these high-frequency components are reduced, resulting in a cleaner and smoother image.
2. Image Sharpening:
Sharpening focuses on enhancing edges and small details by boosting high-frequency

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

information. A high-boost filter does this using the formula

$$H(u,v) = A - H_{lp}(u,v),$$

where $A > 1$ and H_{lp} is the low-pass filter.

3. Boundary Detection:

Edges occur where pixel values change quickly, which corresponds to high-frequency areas.

Using high-pass filters or gradient methods (such as Sobel or Laplacian), these edge regions can be highlighted to produce clear boundary maps.

4. Evaluation:

- Quantitative: Measures like PSNR, MSE, and SSIM help compare image quality.
- Qualitative: Visual aspects such as clarity, sharp edges, and smoothness indicate how well the processing worked.

Code:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from skimage import metrics

img = cv2.imread("/content/reference.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:

    raise ValueError("Image not found. Please check dataset path.")

f = np.fft.fft2(img)

fshift = np.fft.fftshift(f)

magnitude_spectrum = 20 * np.log(np.abs(fshift))

rows, cols = img.shape

crow, ccol = rows // 2, cols // 2

mask = np.zeros((rows, cols), np.uint8)

r = 40

cv2.circle(mask, (ccol, crow), r, 1, thickness=-1)
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
fshift_low = fshift * mask
```

```
img_low = np.fft.ifft2(np.fft.ifftshift(fshift_low))
```

```
img_low = np.abs(img_low)
```

```
A = 1.5
```

```
high_boost = A + (1 - mask)
```

```
fshift_sharp = fshift * high_boost
```

```
img_sharp = np.fft.ifft2(np.fft.ifftshift(fshift_sharp))
```

```
img_sharp = np.abs(img_sharp)
```

```
edges = cv2.Canny(np.uint8(img_sharp), 80, 180)
```

```
_, binary_boundary = cv2.threshold(edges, 50, 255, cv2.THRESH_BINARY)
```

```
titles = ['Original', 'Low-pass Filtered', 'High-boost Sharpened', 'Binary Boundary']
```

```
images = [img, img_low, img_sharp, binary_boundary]
```

```
plt.figure(figsize=(10, 6))
```

```
for i in range(4):
```

```
    plt.subplot(2, 2, i + 1)
```

```
    plt.imshow(images[i], cmap='gray')
```

```
    plt.title(titles[i])
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
mse = metrics.mean_squared_error(img, img_low)
```

```
psnr = metrics.peak_signal_noise_ratio(img, img_low, data_range=255)
```

 Marwadi University Marwadi Chandarana Group	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

```
ssim = metrics.structural_similarity(img, img_low, data_range=255)
```

```
print(f'MSE (Low-pass): {mse:.4f}')
```

```
print(f'PSNR (Low-pass): {psnr:.2f} dB')
```

```
print(f'SSIM (Low-pass): {ssim:.3f}')
```

Output:

Original



Low-pass Filtered



High-boost Sharpened



Binary Boundary



MSE (Low-pass): 535.5783

PSNR (Low-pass): 20.84 dB

SSIM (Low-pass): 0.509

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: DSIP (01CT1513)	All Open Handed	
All Open Handed	Date:	Enrolment No: 92301733046

Conclusion:

In the present study, frequency-domain methods were used for the processing of OPE7 images. The low-pass filter was very effective in minimizing noise as it cut off a significant amount of the high-frequency signals, whereas the high-boost filter worked on the enhancement of the details by milder contrast distribution. The process of extracting the boundaries also resulted in edges that were both clean and well-defined. In conclusion, my findings from the comparison of these filtering methods indicate that the use of frequency-domain processing can simultaneously eliminate noise and keep important features, thus improving the images' clarity and practical usability.